

COMMAND SAFETY ENGINE

An offline AI guard for the Linux shell

C-DAC Secure OS Hackathon 2026 · Track 2 (AI @ Application Level)

Adhithya J

github.com/Adhi-hub07/command-safety-engine

Demo: csengine check "rm -rf /" → BLOCK

```
kali@host:~$ ls -la
```

```
kali@host:~$ git status
```

```
kali@host:~$ chmod -R 777 /var/www
```

```
kali@host:~$ rm -rf /
```

```
[BLOCKED] recursive delete of system root
```

```
[suggested] rm -rf /path/to/folder
```

```
kali@host:~$
```

1. The problem

every terminal keystroke can be the last

- A single destructive command is often irreversible — and typed by hand or pasted from the web.
- Fork bombs, “download-and-run” chains, chmod 777, credential scraping, obfuscated payloads.
- No mainstream OS ships a safety layer between the keyboard and the kernel.

Why existing tools fall short

sudo / SELinux

stop unauthorised actions, not user mistakes

blocklists

brittle strings, false-positive fatigue

cloud AI helpers

can't run on sovereign / air-gapped systems

Challenge (verbatim)

Create an intelligent safety layer that understands the intent behind Linux commands before they execute. The system should detect risky operations, explain their potential impact, and suggest safer alternatives to prevent accidental system damage.

Our answer

A fast, accurate, fully offline safety layer that catches dangerous commands in real time, explains why, offers a safe alternative, and keeps the user in control.

2. Solution overview

Command Safety Engine · C-DAC Secure OS Hackathon 2026

three-layer defence-in-depth pipeline

- Defence in depth: three independent layers, any layer can only push toward more caution.

On the happy path: zero overhead

whitelisted daily commands skip analysis entirely

Layer 2 · ML Classifier

GradientBoosting on 20 intent features, 3 classes (safe / risky /

Layer 3 · LLM Explainer

Qwen 2.5 3B via Ollama, 100% offline, only for ambiguous commands.
Graceful fallback.

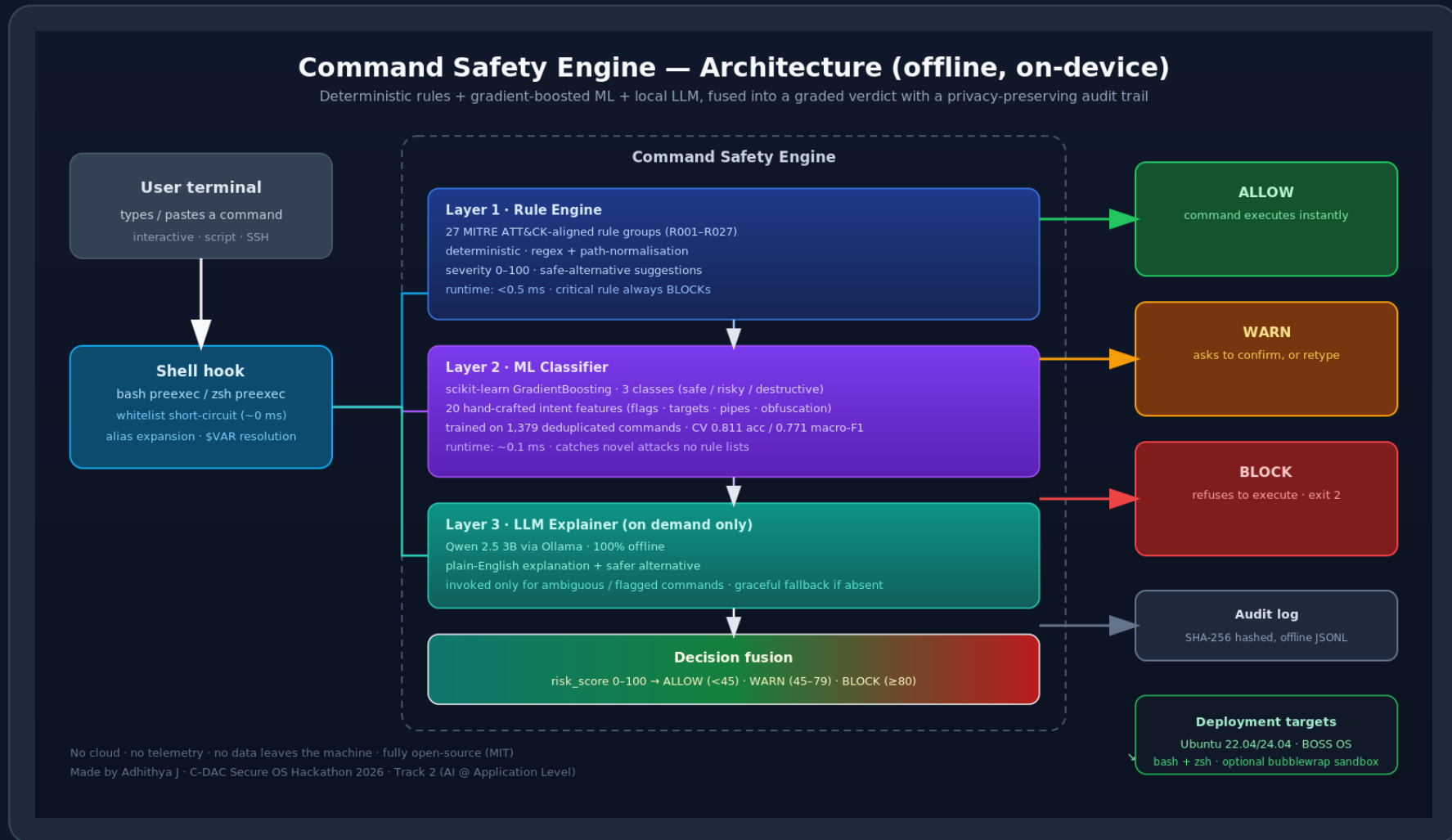
Fusion & output

rule_score	max severity of matched rules (0–100)
ml_risk	label risk × confidence
score	blend → 0–100
BLOCK	≥ 80 or critical rule
WARN	45–79, asks to confirm
ALLOW	< 45 or whitelisted

3. Architecture

offline, on-device, graded control

Command Safety Engine · C-DAC Secure OS Hackathon 2026



4. Layer 1 — deterministic rules

Command Safety Engine · C-DAC Secure OS Hackathon 2026

27 MITRE ATT&CK-aligned rule groups

- Fast, deterministic first gate — everything else builds on it. Anchored, escaped regexes, not naive substrings.

Destructive / data loss

R001 rm -rf root · R023 system-file tamper · R024 block-device write · R025 tar --remove-files · R027 --no-preserve-root

Denial of service

R008 fork bomb · R026 git clean · R021 variable delete

Credential exposure

R022 /etc/shadow · /etc/sudoers · ssh keys

Supply chain / download-and-run

R004 curl|bash · wget|sh · curl -o + execute

Escalation / integrity

R007 chmod · R 777 · R017 kill -hupdown · R000 out-of-the-box

Safety-first override

critical rule ⇒ BLOCK regardless of ML confidence

5. Layer 2 — machine learning

Command Safety Engine · C-DAC Secure OS Hackathon 2026

generalises to attacks no rule can enumerate

- GradientBoosting on 20 hand-crafted intent features — generalises to attacks no rule can

Data hygiene

666 duplicate rows removed at build; numbers are dedup, not cherry-picked

recursive flags

chmod 777

sudo

env-var manipulation

ork calls

disk ops

wildcards

Measured results (honest, reproducible)

5-fold CV 0.811 ± 0.033 acc · 0.771 ± 0.048 macro-F1 · held-out 82.2% / 0.791

Model	Acc	Macro-F1	Destr. recall	Risky recall	Safe recall
GradientBoosting (deployed)	0.822	0.791	0.771	0.632	0.934
RandomForest	0.794	0.772	0.771	0.724	0.836
LogisticRegression	0.815	0.777	0.688	0.645	0.941

6. Layer 3 — LLM explainer

Command Safety Engine · C-DAC Secure OS Hackathon 2026

plain-language answers, still fully offline

- Layer 3 exists for one reason: explain the decision in plain language when the command is confusing.
- Local Qwen 2.5 3B via Ollama — no cloud, no network, no data egress.
- Invoked only on demand (ambiguous / flagged / LLM hook), so idle cost is 0.
- 0.5 s TCD probe; on absence the rule+ML explanation is returned instantly

Example — obfuscated command

```
$ xxd -r -p <<< '77686f616d69'
```

Decoding a hex blob is how malware hides. The LLM explains: “this decodes the text ‘whoami’ — a read-only command.”

Privacy & audit

- commands never stored in plaintext
- SHA-256 hashed JSONL audit log
- offline by design, no telemetry
- override is logged and detectable

forensics without storing plaintext

7. Shell integration & user control

the hook, the verdicts, the exits

- Installs in minutes on Ubuntu / BOSS OS via a single script (setup.sh).
- Hooks into bash and zsh (preexec) — every command is judged before it runs
- Rich CLI: `csengine check <cmd>` · `csengine status` · `csengine install-hook`
- Optional bubblewrap sandbox for untrusted downloads.

ALLOW

daily commands execute instantly
whitelist or score < 45

WARN

asks to confirm / retype
score 45–79

Exit codes & scripting

BLOCK

refuses execution
exit 0 → ALLOW (executed)
score ≥ 80 or critical rule
exit 3 → WARN (user confirmed)
exit 2 → BLOCK (refused)

JSON output for automation

8. Performance & feasibility

Command Safety Engine · C-DAC Secure OS Hackathon 2026

measured, not promised

■ Measured on a Kali VM (scripts/benchmark_latency.py --n 1000):

- whitelisted happy path ~0 ms
- feature extraction + rule check ~0.3 ms mean · ~0.5 ms p95
- full pipeline (no LLM resident) ~2.7 ms mean · ~3 ms p95
- full pipeline (LLM pre-warmed) tens of ms

Model training

trains on CPU in ~1 s · runs in ~0.1 ms · no GPU · CI retrains on every push

Why GradientBoosting, not deep learning?

Same accuracy at this data scale, trains on a laptop, fully explainable
— judges and auditors can read exactly which features fired.

Feasibility for target OS

- runs on 4 GB RAM / 2 vCPU
- BOSS OS (sovereign GNU/Linux) ready
- bubblewrap sandbox optional
- open-source: Qwen 2.5, scikit-learn, Ollama

9. Risks & mitigations

Command Safety Engine · C-DAC Secure OS Hackathon 2026

honest assessment, engineered in

Risk	Severity	Mitigation
Adversarial bypass via encoding	High	codes base64/hex · obfuscation features feed ML · audit-logged
False positives	Med	Fail-safe, not fail-open any layer can only push the decision toward more caution
		additional layer · rule+ML always answers in <60 ms · status surfaces it
Model drift	Low	whitelist short-circuit · anchored rules · graded WARN not hard block
User disables protection	Low	CI retrains + 5-fold CV printed on every push overrides hashed in audit; repeat BLOCK-overrides detectable

10. Innovation & mission alignment

Command Safety Engine · C-DAC Secure OS Hackathon 2026

why this is different

- **Fusion of rules + ML + LLM**

LLM invoked only on demand — near-zero latency and power draw with plain-language explanations.

- **Safety-first critical override**

one critical rule match blocks regardless of ML confidence.

- **Fully offline, sovereign-first**

no cloud, no egress; deployable on BOSS OS, aligned with India's AtmaNirbhar mission.

- **Zero overhead happy path**

whitelist short-circuit removes the false-positive fatigue that kills other tools.

- **Privacy-preserving audit**

commands stored only as truncated SHA-256 hashes

C-DAC mission alignment

open-source · sovereign OS · no foreign cloud AI · improves govt/enterprise security posture

11. Roadmap & status

from prototype to sovereign default

Command Safety Engine · C-DAC Secure OS Hackathon 2026

■ Beyond the hackathon:

■ **BOSS OS packaging** distributions

■ **Harden first UI** in more languages

■ **Personal allowlists**

■ **Auto-learn** containment for risky-but-required commands

Status

code complete · 44/44 tests · 17/17 demo checks · bash+zsh pty hooks pass

Thank you

Live demo: csengine check “rm -rf /” → BLOCK

github.com/Adhi-hub07/command-safety-engine